Knowledge-Based Systems 273 (2023) 110626

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

DI-NIDS: Domain invariant network intrusion detection system

Siamak Layeghy *, Mahsa Baktashmotlagh, Marius Portmann

School of ITEE, The University of Queensland, Brisbane, Australia

ARTICLE INFO

Article history:
Received 15 October 2022
Received in revised form 26 April 2023
Accepted 5 May 2023
Available online 12 May 2023

Keywords:
Adversarial domain adaptation
Network intrusion detection system (NIDS)
Cross-domain evaluation
Domain invariant anomaly detection
One-class SVM

ABSTRACT

The performance of machine learning based network intrusion detection systems (NIDSs) severely degrades when deployed on a network with significantly different feature distributions from the ones of the training dataset. In various applications, such as computer vision, domain adaptation techniques have been successful in mitigating the gap between the distributions of the training and test data. In the case of network intrusion detection however, the state-of-the-art domain adaptation approaches have had limited success. According to recent studies, as well as our own results, the performance of an NIDS considerably deteriorates when the 'unseen' test dataset does not follow the training dataset distribution. In order to enhance the generalisability of machine learning based network intrusion detection systems, we propose to extract domain invariant features using adversarial domain adaptation from multiple network domains, and then apply an unsupervised technique for recognising abnormalities, i.e., intrusions. More specifically, we train a domain adversarial neural network on labelled source domains, extract the domain invariant features, and train a One-Class SVM (OSVM) model to detect anomalies. At test time, we feedforward the unlabelled test data to the feature extractor network to project it into a domain invariant space, and then apply OSVM on the extracted features to achieve our final goal of detecting intrusions. Our extensive experiments on the NIDS benchmark datasets of NFv2-CIC-2018 and NFv2-UNSW-NB15 show that our proposed setup demonstrates superior cross-domain performance in comparison to the previous approaches.

1. Introduction

For network anomaly/intrusion detection, labelling millions of real-world network records requires a significant amount of resources and human expertise. Various attacks do not happen all the time/everywhere, and due to privacy and security concerns, labelled real-world network intrusion detection systems (NIDS) datasets are scarce, and rarely publicly available. Accordingly, the common way of training machine learning (ML) based NIDSs is by using publicly available synthetic datasets. However, as has been shown [1], there is a considerable difference in the feature distribution of the benign/background traffic between real-world datasets and the synthetic benchmark datasets created in research labs. Thus, the *cross-domain* performance, i.e., the capability of correctly classifying the test samples in the presence of distribution shifts, is essential for adapting ML-based NIDSs for successful deployment and use in real-world production networks.

However, the majority of the proposed ML-based NIDSs are evaluated only on *domain-specific* datasets, i.e., the training and evaluation samples are drawn from the same dataset, and cross-domain evaluation is rarely considered.

Moreover, current approaches for anomaly detection assume similar feature distributions for the training and test datasets [2]. Therefore, these models fail to perform well when there is a distribution difference between the train (i.e., source) and test (i.e., target) data. Our study shows that the performance of the existing NIDS models degrades when they are applied in a network environment that has a different feature distribution compared to the training environment/dataset [3,4].

To address the domain shift problem, several domain adaptation (DA) techniques have been introduced in the literature. These techniques try to reduce the gap between the feature representations of the labelled source and unlabelled target domains, so that the classifiers trained on the source domain perform similarly well on the target domain [5,6].

Generally speaking, domain adaptation approaches follow a supervised learning strategy, and thus, they are better suited to class-balanced datasets. However, in the field of network intrusion/anomaly detection, anomalies are relatively rare events, i.e., network anomalies are in high class imbalance compared to the benign/background traffic class. Based on our experimental results, current domain adaptation techniques based on a supervised learning strategy perform poorly on finding anomalies.

* Corresponding author.
E-mail addresses: siamak.layeghy@uq.net.au (S. Layeghy), m.baktashmotlagh@uq.edu.au (M. Baktashmotlagh), marius@ieee.org (M. Portmann).

https://doi.org/10.1016/j.knosys.2023.110626

To address this gap, we are proposing a new unsupervised-learning scheme for cross-domain anomaly detection. In this method, we use a domain adaptation technique to first extract a domain-invariant representation of the data, and then apply the anomaly detection on the projected representation. For the domain adaptation, we use a *Domain-Adversarial Neural Network (DANN)* [7], which is one of the well-known and best working approaches. The DANN can be simply incorporated in the feature extraction network by adding a gradient reversal layer to minimise the difference between the representations of the source and target domains. We first train the DANN using the labelled source data and unlabelled target data, and then, we exploit the feature extraction branch of the DANN to obtain the domain invariant features. Finally, we apply *one-class Support Vector Machines or One-Class SVM (OSVM)* [8] on the extracted features to reach our final goal of cross-domain anomaly detection.

A network anomaly is a somewhat ill-defined concept that is used to describe any networking event such as network attacks/intrusions, failure, etc., that can significantly change the overall and flow-based feature statistics of a monitored network, e.g. the number of input bytes [9].

Note that, similar to other machine learning approaches, OSVMs fails to perform well in the presence of distribution shift. Therefore, before feeding the data to an OSVM, we project the training and test data to a common subspace using a DANN. Our results clearly indicate that projecting features to a domain invariant feature space, before feeding them into an OSVM, significantly improves the cross-domain performance of intrusion detection.

In summary, we propose a *domain-invariant NIDS (DI-NIDS)* framework by leveraging recent advances in the domain adaptation literature. DI-NIDS takes into account the intense class-imbalance nature of anomalies in the NIDS data when addressing the domain shift between the train and test datasets. The proposed framework is evaluated against various ML-models in both cross-domain and domain-specific evaluation scenarios, where it shows superior performance. In the rest of this paper we first discuss the related works in the next section, then explain the proposed solution in Section 4. Extensive evaluation of DI-NIDS and comparison to the state-of-the-art are discussed in Section 5, and Section 6 concludes the paper.

2. Related work

New NIDS proposals such as [10], which focus on the domain-specific intrusion detection problem, have been provided by the research community. For the relevant related works of this paper we considered proposed NIDSs which at least follow a partial cross-domain evaluation approach across different datasets. We managed to find two main groups of NIDS proposals in which some aspects of the training and evaluation datasets are different. While many of these works cannot be directly compared to our work, we still included them since they consider partial elements of cross-domain evaluation.

2.1. Separate source and target domains

In the first group, cross-domain evaluation is realised via separate training (source domain) and test datasets (target domain). In this group of NIDSs, techniques such as domain adaptation and transfer learning are applied on the source domain to acquire the knowledge of anomalies/attacks, and to extend this knowledge to classify samples from the target domain. This group can be further divided into two sub-categories; those which do not require labelled data from the target domain, and those that need a subset of the target domain to have labels. We found three studies from the first subcategory and two studies from the second subcategory as explained below.

In the first subcategory, consisting of studies that do not need labelled data from the target domain, [11] is the only work, to the best of our knowledge, that uses a DANN in the field of network intrusion detection. The paper focuses on detecting attacks in smart grid networks. By applying the adversarial training to adapt learned models on normal operation data of the ISO New England grids, the authors try to detect attacks at different times of the day on their smart grid network. The paper shows that, due to the load demand changes during different times of day, conventional ML-based NIDSs fail to detect attacks, and their proposed DANN-based method improves the detection performance. The authors use false data injection attacks synthesised on the IEEE 30-bus system for the evaluation of their proposed framework. The paper shows that the proposed method has superior detection performance for persistent threats recurring in a highly dynamic smart grid, compared to conventional ML-based NIDSs.

In [12] partial domain adaptation is used to map the source and target domains to a domain-invariant feature space to address the differences between the source and target datasets. The authors use weighted adversarial networks-based domain adaptation for transferring knowledge from the publicly available labelled datasets, such as CIC-IDS2017 [13], to an unlabelled Internet of Things (IoT) dataset, such as [14]. In order to evaluate their proposed framework, the authors apply it to a combination of benign traffic and various common attack classes, across two datasets. They train their model on CIC-IDS201, and evaluate it on the IoT dataset. In another evaluation, they train and test their model on different attack classes to evaluate the performance of their model for detecting unknown attacks. The authors finally compare their framework with a DANN on a binary classification problem consisting of benign traffic and a specific attack class, and show it performs similar to a DANN. While this is an example of a partial cross-domain evaluation of NIDSs, they do not evaluate their methods on full set of classes of the datasets. In addition, the evaluation only considers one direction of the cross-domain evaluation, i.e. with training on one dataset and evaluation on the other, but not vice versa. As we show in this paper, this is a significant limitation, since the results are highly asymmetric.

The authors of [15] propose the Energy-based Flow Classifier (EFC), an anomaly-based classifier that infers a statistical model based on labelled benign samples. They define the concept of *quantum energy* for network flows and compute a threshold for the benign flows as the 95th percentile of the benign flows' energy distribution. Then, they compare the energy of a given flow to this threshold and declare a flow as malicious if its energy is above the threshold. The authors use three versions of the CIC-IDS [13] dataset for the evaluation of their proposed algorithm. The paper's results are compared with conventional ML-based NIDSs for both domain-specific, i.e., the same dataset as the source and target, and cross-domain, i.e., different source and target domains. While the reported domain-specific performance is relatively high, the cross-domain performance is significantly reduced.

While these previous studies do not require labelled data from the target domain, the next two studies need a small portion of the target domain to have labels. The first paper in this group [16] considers a host-based intrusion detection approach, rather than network intrusion detection, and aims to reduce the number of labelled samples from the target domain. The authors use two different host-based intrusion detection datasets as the source and target domains respectively. By using fine tuning techniques of deep learning models, such as freezing the hidden layers, they manage to reduce the number of labelled samples from the target domain, while improving the Area Under Curve (AUC) metric by 8%.

The last paper [17] in this subcategory uses domain adaptation to address the scarcity of labelled training data by transferring the acquired knowledge from a publicly available labelled dataset. Initially, the authors use the UNSW-NB15 [18] dataset and divide it into two subsets of complementary attack classes, and evaluate their approach for the same feature set assessment. Then they use the NSL-KDD [19] dataset as the source and the UNSW-NB15 [18] dataset as the target domain, and evaluate their proposed method for different feature set assessment. Based on the provided results, the proposed method achieves a higher accuracy for various attack classes compared to the fine tuning method, for the same number of samples from the target domain. Although these two methods cannot be directly compared to our work, as they need labels from the target domain, they discuss the challenges of addressing the distribution gap between different intrusion detection datasets.

2.2. Partially different domains

In the second group of studies, the training and test datasets are the same, but the dataset is divided into subsets containing various attack classes. One subset is used as the source and the other, which might include attack classes not present in the source domain, as the target domain. Techniques such as domain adaptation and transfer learning have been used to transfer the knowledge learnt from the attack classes available in the source domain, to classify/detect the attack classes in the target domain. It is noticeable that although there are studies that employ transfer learning for closely related tasks such as malware detection [20], this paper primarily focuses on research concerning network intrusion detection.

In [21], a transfer learning algorithm is used to transfer the knowledge of an image-based representation of network flows. The authors train a convolutional neural network (CNN) in the source domain and augment it with one dense layer in the target domain. In [22] the authors also use a CNN architecture for transfer learning on NIDS datasets. They use two concatenated CNNs to learn network attack patterns on a divided NSL-KDD dataset [19] and improve unknown/unseen attack detection performance in comparison to conventional ML-based NIDSs. Similar to [21], the authors in [23] convert divided KDD99 dataset [24] records into gray-scale images which are then processed for detecting attacks using a CNN architecture. For the purpose of transfer learning, they use samples of unseen attacks to fine tune the trained CNN.

With the exception of [15], none of the related works discussed in this section provides a comparable cross-domain evaluation of the proposed NIDS across different benchmark datasets. While the methods proposed in [16,17] apply domain adaptation techniques on network intrusion detection datasets, they are fundamentally different to our approach, since they rely on the availability of target domain labels, which are difficult to obtain in real-world networks. In contrast, our method proposed in this paper does not require target domain labels, and is hence much more practical.

The last three discussed studies, i.e., [21–23], do not consider domain adaptation, and mainly focus on detecting one or more attacks that were unseen during training.

While [11,12] are similar to our work in the sense that they do not require any target domain labels, these works do not provide a complete cross-domain evaluation such as presented in this paper. In [11], the proposed method is evaluated against the attacks injected into a smart grid network, and there is no consideration of applying such a method on the publicly available NIDS datasets. In [12] the proposed method is evaluated against subsets of the target domain and performance metrics are provided for individual attack classes.

The method presented in [15] is the only approach with a cross-domain performance evaluation that is comparable to ours. However, the performance of the method proposed in [15] shows a 19% degradation (on average) of the cross-domain performance compared to the corresponding domain-specific performance. As we will demonstrate, our proposed method performs significantly better in this critical regard.

3. Background: Unsupervised domain adaptation

Various unsupervised domain adaptation (UDA) proposals exist, which employ different techniques to mitigate the distributional differences between the source and target datasets.

Domain Alignment Networks (DAN) [25] is a UDA that utilises a domain alignment layer to align the feature distributions of the source and target domains and learn domain-invariant features. The method employs the Maximum Mean Discrepancy (MMD) loss, which measures the distance between the means of the feature distributions, to encourage the model to learn domain-invariant features that can generalise well across different domains. However, DAN solely focuses on aligning the means of the feature distributions of the source and target domains using MMD loss, which may not always be adequate to capture complex domain shifts. Furthermore, the method does not explicitly take into account the class information of the source and target domains during the alignment process. This may lead to sub-optimal feature alignment in scenarios where the class distributions of the source and target domains differ significantly.

Adversarial Discriminative Domain Adaptation (ADDA) [26] is another UDA that combines adversarial training with a discriminative loss to acquire domain-invariant features. It is based on the Generative Adversarial Networks (GANs) principle, with two main components - a feature extractor and a domain discriminator. The adversarial training alternates optimising the feature extractor and domain discriminator to prevent distinguishing source and target domain features. By jointly optimising the adversarial loss and classification loss, ADDA learns domain-invariant features that generalise well across different domains. This method has achieved impressive results in several unsupervised domain adaptation tasks like object recognition and semantic segmentation. However, the adversarial training can be sensitive to hyperparameter choices and may suffer from instability during training. Moreover, the method does not explicitly consider the class information during the alignment process, which could lead to misaligned classes.

Maximum Mean Discrepancy (MMD) [27] is a kernel-based method UDA that aims to minimise distribution discrepancy between source and target domains by comparing their means in a Reproducing Kernel Hilbert Space. It maps data into a common feature space using a kernel function, and computes MMD as a distance measure between the means of source and target domain data. This method is effective and widely used in unsupervised domain adaptation tasks. Nonetheless, the method may not be efficient for large datasets due to its computational complexity. In addition, it assumes that the two datasets are drawn from continuous distributions, which may not be applicable in real-world scenarios where data is discrete or contains missing values. Furthermore, it assumes that the kernel function is fixed and known in advance, which may be challenging to choose and can significantly impact the results of the test.

Cycle-Consistent Adversarial Domain Adaptation (CyCADA) [28] combines the ideas of cycle-consistency and adversarial training to learn domain-invariant features. In CyCADA, the goal is to learn a mapping between the source and target domains such that the mapped source domain data is indistinguishable from the target domain data, and vice versa. The key innovation of CyCADA is the

introduction of a cycle-consistency loss, which ensures that the mappings between the source and target domains are consistent and reversible. This cycle-consistency loss is combined with an adversarial loss, which encourages the mapped source domain data to be indistinguishable from the target domain data. The main limitation of CyCADA is its reliance on cycle-consistency, which can be sensitive to hyperparameter choices and may not always guarantee meaningful mappings between domains. Moreover, the method can be computationally expensive due to the need for training multiple networks.

Deep CORAL [29] aligns the second-order statistics of the source and target domain feature distributions using a deep neural network. It introduces a CORrelation ALignment (CORAL) loss to measure the distance between source and target domain covariance matrices and adds it to the standard classification loss. Deep CORAL can be easily integrated into existing deep learning architectures, enabling the neural network to learn domain-invariant features that generalise well across different domains. However, merely aligning the second-order statistics of source and target domain feature distributions may not be adequate to capture intricate domain shifts.

Contrastive Adaptation Network (CAN) [30] learns domain-invariant features by minimising the contrastive loss between positive pairs (samples from the same class) and negative pairs (samples from different classes) across domains. The CAN consists of three main components: a feature extractor, a domain discriminator, and a label predictor. The feature extractor learns domain-invariant features, the domain discriminator distinguishes between source and target domains, and the label predictor performs classification. The effectiveness of CAN is demonstrated on several benchmark datasets, showing that it outperforms existing unsupervised domain adaptation methods. Nonetheless, CAN relies heavily on the quality of the negative pairs for contrastive learning. If the negative pairs are not representative of the true data distribution, the model may not learn meaningful domain-invariant features.

More recently, Generative Domain Adaptation (GDA) [31], is proposed to use domain adaptation for face anti-spoofing. It consists of three main components: a generator, a discriminator, and a face anti-spoofing classifier. The generator is responsible for generating target domain samples, the discriminator distinguishes between real and generated samples, and the classifier performs face anti-spoofing. The GDA framework is trained in two stages. First, the generator and discriminator are trained using adversarial learning to generate realistic target domain samples. Second, the face anti-spoofing classifier is trained using the generated samples and the source domain data. The reliance on GANs is the main limitation of GDA, as it can be challenging to train and may suffer from mode collapse, leading to less diverse generated samples.

Another recent study, Unsupervised Domain Generalisation (UDG) [32], tries to address three challenges of (1) the presence of domain-specific biases, (2) the difficulty in learning domain-invariant features, and (3) the lack of a clear evaluation protocol for assessing the performance of domain generalisation methods. The UDG framework consists of two main components: a feature extractor and a domain classifier. The feature extractor learns domain-invariant features, while the domain classifier distinguishes between different source domains. The authors introduce a new loss function that combines classification loss and domain confusion loss to encourage the learning of domain-invariant features. Nonetheless, the UDG framework may struggle if the features learned by the feature extractor are not truly domain-invariant, the model may not generalise well to unseen target domains

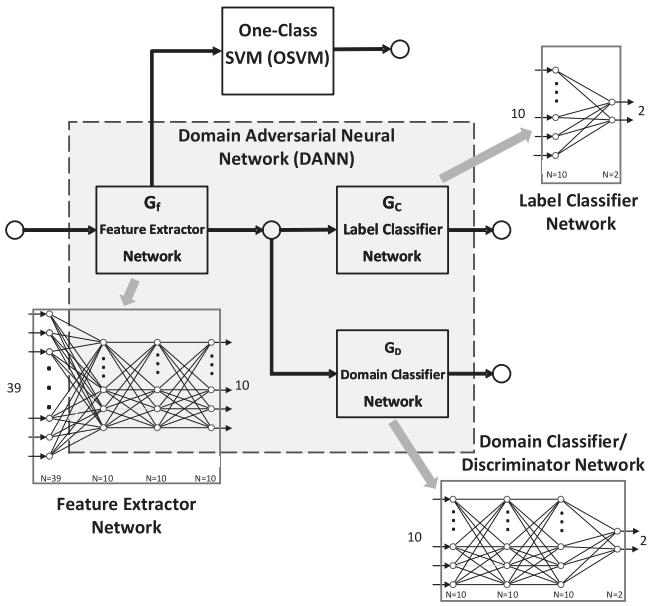


Fig. 1. Proposed DI-NIDS architecture.

The above-mentioned approaches are suitable for class-balanced datasets, but may not be effective in dealing with imbalanced datasets. In the context of network intrusion and anomaly detection, anomalies are rare events, leading to high class imbalance compared to benign or background traffic. Hence, these methods are not optimal for detecting anomalies in networking data. To address this gap, a new method is proposed in the next section.

4. Proposed method (DI-NIDS)

Fig. 1 shows the architecture of *Domain Invariant-Network Intrusion Detection System (DI-NIDS)*, our proposed approach, consisting of two main components, DANN and OSVM. While OSVMs generally perform well for one-class classification problems, in particular anomaly detection, they do not perform well for cross-domain data [33] in general. DANNs on the other hand, were designed to address the problem of domain gap, i.e., different feature distributions, in conventional machine learning models. The key idea of DI-NIDS is to enhance the domain adaptability of OSVM by leveraging the capabilities provided by a DANN.

As can be seen in Fig. 1, a dense Multi Layer Perceptron (MLP) is used as the basis of the DANN in our model, with the main hyper-parameters listed in Table 1. As per [7], it is possible to implement a DANN using any feed-forward neural network architecture, and the type of the neural network can be selected to best match the attributes of the input data. For instance, if the input type is an image, convolutional neural networks are typically chosen. Since the NIDS data consist of a small number of numeric and categorical features, we select a basic MLP network as a starting point.

The training of our DI-NIDS follows a two-step process. In the first step, the DANN is trained using the source data, source labels, target data, and domain labels (labels that identify which domain the input belongs to, i.e., train or test dataset). Note that no target class labels are required in the training stage of the DANN. Once the training stage of the DANN is completed, we employ the trained feature extractor network G_f to extract domain invariant features from the data. In the second step, the OSVM is trained using the extracted domain invariant features.

Table 1
Parameters of three neural networks utilised in the implemented DI-NIDS architecture.

Name	Function	Input nodes	Output nodes	No. of hidden layers	Hidden layers nodes
G_f	Feature extractor	39	10	2	10
G_C	Label classifier	10	2	0	0
G_D	Domain classifier	10	2	1	10

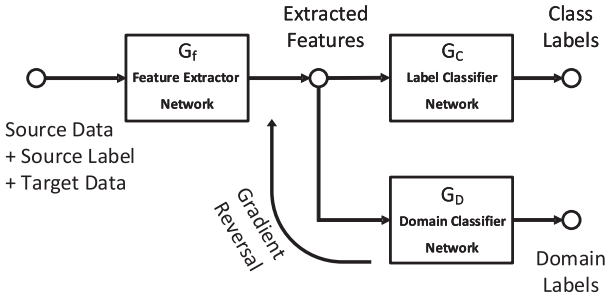


Fig. 2. Domain adversarial neural network.

DANN Component: In ML-based NIDSs, as is the case in other application areas as well, it is difficult and time-consuming to label real-world data. Consequently, synthetic datasets are often used for training and evaluation of machine learning models. However, these synthetic datasets usually do not adequately represent real-world networks and suffer from distribution shift, i.e. a significant difference in feature distributions [1]. Generally speaking, *domain adaptation* aims at as making the distributions of source and target domains similar, so that if a classifier or detector is trained on the source data, it can perform well on the target data. [7].

In this work, we employ adversarial domain adaptation to extract domain invariant features from the source and target domains. The architecture of a DANN [7] is shown in Fig. 2. The architecture consists of three networks: *feature extractor* to extract features from the source and target domains, *label classifier* to predict class labels, and *domain classifier* to predict domain labels. The domain classifier network includes a gradient reversal layer to make the distributions of the source and target features similar. Specifically, for the samples that are correctly classified by the domain classifier, a penalty is applied through multiplying their gradient by a negative factor during back propagation [7,11].

Definition. Assume $X = \{x_1, x_2, \ldots, x_n\}$ represents the input space, $Y = \{0, 1\}$ is the set of binary labels, and $\eta : X \rightarrow Y$ is a binary classifier. Given the source domain, $\mathcal{D}_S = \{(x_1^S, y_1^S), (x_2^S, y_2^S), \ldots, (x_{n_S}^S, y_{n_1}^S)\}$, and the target domain, $\mathcal{D}_T = \{(x_1^T, y_1^T), (x_2^T, y_2^T), \ldots, (x_{n_T}^T, y_{n_2}^T)\}$, the feature extractor neural network G_f can be defined as

$$G_f(\mathbf{x}; \mathbf{W}, \mathbf{b}) = sigmoid\,(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{1}$$

where $(\mathbf{W}, \mathbf{b})$ are network weights and biases, with $x \in D_S$. The label classifier network G_C can be written as

$$G_C(G_f(\mathbf{x}); \mathbf{V}, c) = sigmoid\left(\mathbf{V}G_f(\mathbf{x}) + c\right) \tag{2}$$

with $(\mathbf{V}, c)$ representing the network parameters. Finally, the domain classifier network G_D can be formulated as

$$G_D(G_f(\mathbf{x}); \mathbf{u}, z) = sigmoid\left(\mathbf{u}^T G_f(\mathbf{x}) + z\right) \tag{3}$$

with $(\mathbf{u}, z)$ the network parameters.

More specifically, for a given sample-label pair (x, y), $x \in X, y \in Y$, the domain classifier loss $L_D(x, \gamma)$ is defined as follows:

$$L_D(x_i, \gamma_i) = \gamma_i log \frac{1}{G_D(G_f(\mathbf{x_i}))} +$$

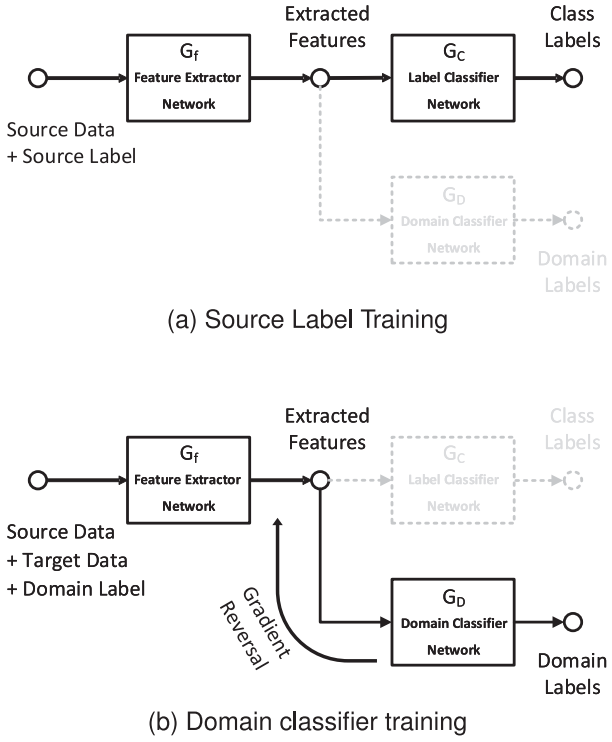


Fig. 3. Two sub-processes of DANN including: (a) Source classifier training and (b) Domain classifier training.

$$(1 - \gamma_i) log \frac{1}{1 - G_D(G_f(\mathbf{x_i}))} \tag{4}$$

where

$$\begin{cases} \gamma_i = 0 & if \quad x_i \in \mathcal{D}_S \\ \gamma_i = 1 & if \quad x_i \in \mathcal{D}_T \end{cases}$$

With the label classifier loss $L_y(x, y)$ defined as

$$L_y(x_i, y_i) = log \frac{1}{G_C(G_f(\mathbf{x_i}))_{y_i}} \tag{5}$$

the optimisation function of the domain adversarial neural network [7] can be written as

$$\min_{W,b,V,c,u,z} \left[\frac{1}{n_S} \sum_{x \in \mathcal{D}_S} L_y(x, y) - \frac{\lambda}{n_S} \sum_{x \in \mathcal{D}_S} L_D(x, \gamma) - \frac{\lambda}{n_T} \sum_{x \in \mathcal{D}_T} L_D(x, \gamma) \right] \tag{6}$$

with n_S and n_T being the number of samples from the source and target domains.

Two simultaneous sub-processes of *label classifier training* and *domain classifier training*, as shown in Fig. 3-(a) and (b) respectively, contribute to the training process of the DANN. During the label classifier training, the source data and its labels are passed through the feature extractor (G_f) and label classifier (G_C)

networks, and are optimised through stochastic gradient descent to update the weights in G_C and G_f.

In the domain classifier training sub-process, the source data, the target data, and *domain labels* are passed through the feature extractor (G_f) and domain classifier (G_D). The domain labels (γ) identify the domain to which a given input belongs, as defined in Eq. (4). In this sub-process, the samples with correctly predicted domain labels are penalised by the "*Gradient Reversal*" layer. The two sub-processes are optimised simultaneously, so the domain invariant and discriminative features can be learnt.

OSVM Component: In the one-class classification problem, the objective is to learn the feature distributions of the normal/benign network flows, and identify samples that deviate significantly from that distribution. This is known as anomaly detection.

Support Vector Machines (SVM) [34] were originally proposed for the multi-class classification problems and were later adopted for the one-class classification problem as proposed in [8,35].

Assume $\chi = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ represents a dataset in which $x_i \in \mathbb{R}^d$ is the ith data point in a d-dimensional input space $\mathbb{I}$, and $y_i \in \{-1, 1\}$ represents the ith output, i.e., class labels. A SVM uses a nonlinear function ϕ, i.e., a *kernel*, to project data points from their input space $\mathbb{I}$ to a high dimensional feature space $\mathbb{F}$, in which the classes can be distinguished by a linear hyperplane $w^T x + b = 0$, with $w \in \mathbb{F}$ and $b \in \mathbb{R}$. To find this hyperplane, SVM minimises the following objective function [35]:

$$\min_{w,b,\xi_i} \left[\frac{\|w\|^2}{2} + C \sum_{i=1}^{n} \xi_i \right] \tag{7}$$

subject to the two constraints of:

$$\begin{cases} y_i \left(w^t \phi\left(x_i\right) + b \right) \geq 1 - \xi_i & \forall\, i = 1, \ldots, n \\ \xi_i \geq 0 & \forall\, i = 1, \ldots, n \end{cases}$$

Here, C is a constant determining the number of training data points within the margin between two classes, i.e., training error, and ξ is the *slack variable* to prevent overfitting. Solving this minimisation problem via Lagrange multipliers results in the following classification rule for a given data point x [35]:

$$f(x) = sgn \left[\sum_{i=1}^{n} \alpha_i y_i K\left(x, x_i\right) + b \right] \tag{8}$$

with α_i being the Lagrange multipliers and the function $K(x, x_i) = \phi(x)^T \phi(x)$ being the *kernel function*.

In OSVM [8], instead of learning a hyperplane to separate two classes of data, a hyperplane is learnt to separate the abnormal data points from the normal density in the origin. Hence, the intention is to maximise the distance of the learnt hyperplane from the origin in the feature space $\mathbb{F}$. An OSVM can be mathematically formulated as a minimisation of the below objective function [35]:

$$\min_{w,\xi_i,\rho} \left[\frac{\|w\|^2}{2} + \frac{1}{\nu n} \sum_{i=1}^{n} \xi_i - \rho \right] \tag{9}$$

subject to the following two constraints:

$$\begin{cases} \left(w.\phi\left(x_i\right)\right) \geq \rho - \xi_i & \forall\, i = 1, \ldots, n \\ \xi_i \geq 0 & \forall\, i = 1, \ldots, n \end{cases}$$

with $\nu \in (0, 1)$ being the upper bound identifier for the fraction of outliers and lower bound on the support vectors, and $\rho \in \mathbb{R}$ being the offset. The Lagrange method is used to solve the above minimisation problem which results in the following classification rule [35]:

$$\begin{aligned} f(x) &= sgn\left(w\phi\left(x_i\right) - \rho\right) \\ &= sgn\left[\sum_{i=1}^{n} \alpha_i y_i K\left(x, x_i\right) - \rho \right] \end{aligned} \tag{10}$$

The hyperplane identified by w and ρ has the maximum distance to the origin in the feature space $\mathbb{F}$, which separates anomalous data points from the normal ones concentrated in the origin.

5. Experimental evaluation

5.1. Datasets

For the evaluation of our proposed approach and models, we used the NetFlow versions of two publicly available NIDS datasets, UNSW-NB15 [18] and CIC-2018 [13]. These are the two most-cited NIDS benchmark datasets among the recent NIDS datasets, providing a more realistic representation of today's network traffic in comparison to older benchmark datasets such as NSL-KDD [19].

The original UNSW-NB15 dataset was generated and published by researchers at the University of New South Wales at the Australian Defence Force Academy Canberra in 2015. The second dataset, CIC-2018, was collected from a completely different network setup and published by the Canadian Institute for Cybersecurity (CIC), University of New Brunswick (UNB), in 2018.

The original versions of these datasets are published with two very different feature sets, with only six common features across the two sets, out of a total of 42 and 75 features of UNSW-NB15 and CIC-2018 datasets respectively [36]. This generally makes it impossible to fairly compare the performance of ML models on both datasets.

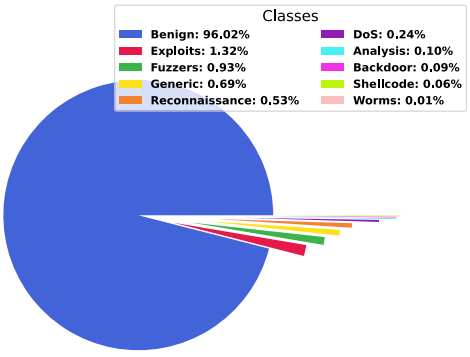
Therefore, a common feature set was needed for the cross-domain evaluation of our ML models. Accordingly, we used the NFv2-UNSW-NB15 and NFv2-CIC-2018 datasets, that were converted to NetFlow from their original formats in [37]. NetFlow is the *de facto* standard in network flow reporting and is widely deployed in real-world networks. The features in the NetFlow versions of these datasets are comprised of 43 NetFlow version 9 fields, which represent bi-directional network flows.[1]

Fig. 4 shows the class distributions for these two datasets. The NFv2-UNSW-NB15 dataset includes 2,390,275 flows, labelled as either Benign/background traffic, or one of the 9 attack classes. The Benign class makes up 96.02% of the entire dataset. The NFv2-CIC-2018 dataset consists of 18,893,708 flows, which are either Benign or belong to one of the 14 attack classes, including various DoS and DDoS attacks, SQL Injection, Infiltration and Brute Force attacks. In this dataset, 88.05% of the flows are Benign.

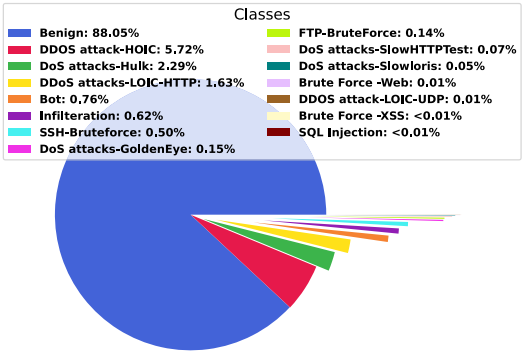
Since the focus of this paper is on binary classification, all the various attack classes in each dataset are aggregated into a single class called **Attack**. However, the nature of this Attack class is totally different for each dataset. Indeed, the number of attack types, the number of flows belonging to each attack type and the ratio of number of flows in each attack type to total flows are different in each dataset. Even the attacks with similar names from two datasets, such as *DoS* (from NFv2-UNSW-NB15) and *DoS attacks-Slowloris* (from NFv2-CIC-2018) represent completely different types of attacks.

Consequently, these datasets not only represent different network environments, as they have been generated in completely

[1] A previous version of these datasets with 20 NetFlow fields is also publicly available [36].



(a) NFv2-UNSW-NB15



(b) NFv2-CIC-2018

Fig. 4. Class distribution for the datasets used in this study (a) NFv2-UNSW-NB15 and (b) NFv2-CIC-2018.

different networks, they represent domains with different label sets. This is also reflected in previous studies such as [4], where it is shown that the performance of conventional ML models trained on one of these datasets severely degrades when tested on the other dataset. Another previous study [1] has also shown the difference between the feature distributions in the benign/background traffic of various NIDS datasets.

5.2. Domain invariant projection

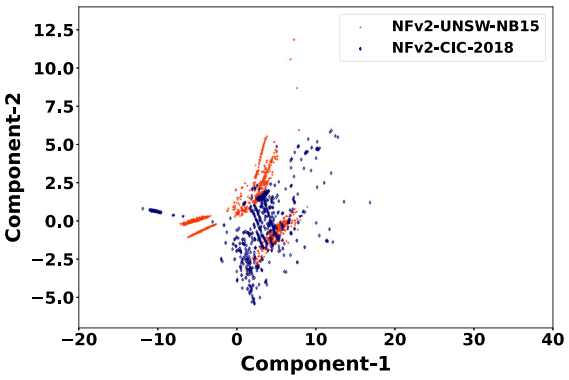
As shown in [38], the main objective of adversarial domain adaptation (ADA) is to map datasets to a new feature space, where:

- The feature distributions of the two datasets are aligned
- The feature distributions of different classes are discriminative

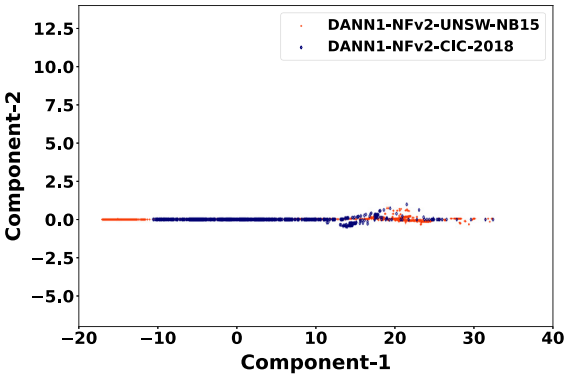
In this section, we visually explore these objectives for the two NIDS datasets used in this study. First, we investigate the separation or closeness of feature distributions in these datasets before and after applying domain-invariant projection via DANN. Then, we examine the feature distributions of different classes before and after the projection using DANN.

Fig. 5 illustrates the visualisation of the NFv2-UNSW-NB15 and NFv2-CIC-2018 datasets before and after projection via DANN. In Fig. 5-(a), the two datasets are shown before projection, while in Fig. 5-(b) and (c), they are shown after projection via DANN (G_f network) trained on NFv2-UNSW-NB15 (referred to as DANN1) and NFv2-CIC-2018 (referred to as DANN2) datasets, respectively. The input datasets consist of 43 features, and after passing through the Feature Extractor (G_f) of the DANN component, the number of features is reduced to 10, as indicated by the number of nodes on the output layer of G_f in Fig. 1. To generate these visualisations, we use the Isomap [39] embedding algorithm to reduce the dimensionality of the data from 43 and 10 to 2, respectively.

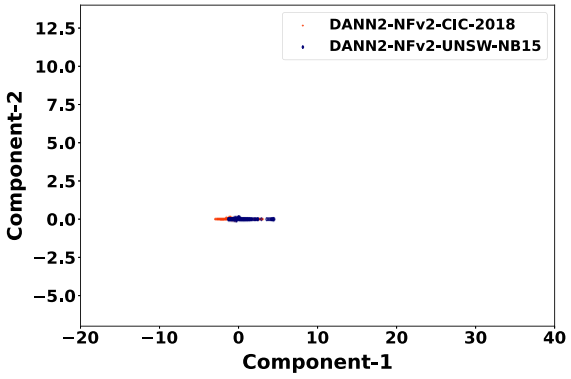
Let us now turn our attention to the second objective with respect to the NIDS datasets. Fig. 6 presents the visualisation of the Attack and Benign classes of the NFv2-CIC-2018 dataset, both before and after undergoing DANN projection. To generate Fig. 6-(a), (b), and (c), we have zoomed in on Fig. 5-(a), (b), and (c)



(a) Two datasets before projection



(b) Two datasets after projection via DANN-1



(c) Two datasets after projection via DANN-2

Fig. 5. Domain shift for NFv2-UNSW-NB15 and NFv2-CIC-2018 datasets in (a) Before projection (Input data), and when projected via model trained on (b) NFv2-UNSW-NB15 (DANN1) and (c) NFv2-CIC-2018 (DANN2).

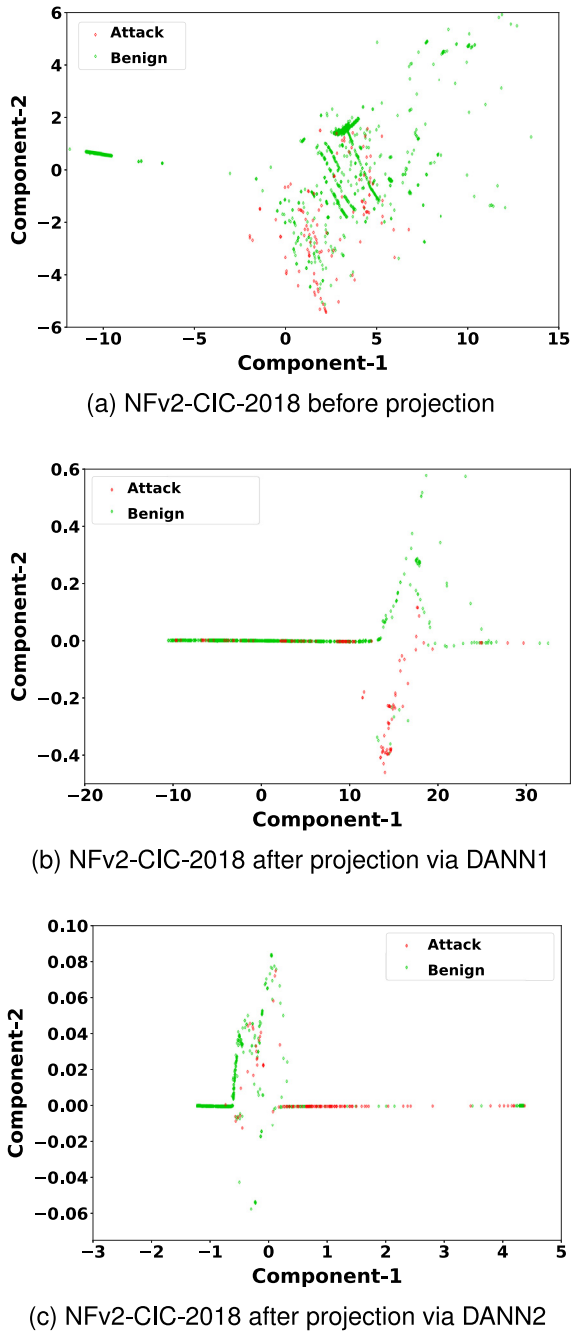


Fig. 6. The Attack/Benign separation for the NFv2-CIC-2018 dataset for (a) input data before projection, and after projection via the model trained on (b) NFv2-UNSW-NB15 (DANN1) and (c) NFv2-CIC-2018 (DANN2) datasets.

respectively, and selected the NFv2-CIC-2018 dataset to make it easier to discern the changes before and after projection. We have utilised green and red colours to represent the Benign and Attack classes, respectively. Therefore, Fig. 6-(a), (b), and (c) depict a portion of the data displayed in Fig. 5-(a), (b), and (c), respectively, to facilitate the investigation of the class distributions.

The visual representation in Fig. 5-(a) reveals that there is minimal overlap between the NFv2-UNSW-NB15 and NFv2-CIC-2018 datasets within the feature space, and the samples appear scattered across a vast area. However, after undergoing projection via either DANN1 (Fig. 5-(b)) or DANN2 (Fig. 5-(c)), a significant number of samples from both datasets are closely clustered together in the feature space. Therefore, it is evident that the first objective of adversarial domain adaptation for the NIDS datasets has been successfully accomplished.

Likewise, the distribution of Attack and Benign classes of the NFv2-CIC-2018 dataset illustrated in Fig. 6-(a) indicates a mixed scatter of both classes, with samples from both classes located in close proximity to one another in many areas of the feature space. There exist only a few scattered clusters of the Benign class. However, once again, after undergoing projection via either DANN1 (Fig. 6-(b)) or DANN2 (Fig. 6-(c)), an increased separation between the two classes can be observed, indicating a potential for improved anomaly detection in the projected datasets.

5.3. Results

In order to evaluate the proposed DI-NIDS framework, we performed three sets of experiments. In the first set of experiments, DI-NIDS was evaluated in a domain-specific setup on our chosen two benchmark datasets, i.e., it was trained and tested on the same dataset, for each dataset separately. In the second and third sets of experiments, we evaluated DI-NIDS in a cross-domain setup, i.e., with training on one dataset and testing on the other. First, one dataset was used as the train/source, and the other dataset as the test/target dataset. Then, we swapped the train/source and test/target datasets and ran the evaluation again, in order to obtain the cross-domain performance in both directions. The domain-specific evaluation mainly serves as a baseline, i.e., to compare the performance to the cross-domain evaluation scenario.

Since there is no previous study of domain adaptation using the same NIDS datasets that we are using in this study, we rely on the results published in one of our previous works [4], which includes both cross-domain and domain-specific evaluation of conventional deep and shallow machine learning models. The models used in this study include a long short-term memory (LSTM) model, an AutoEncoder-based anomaly detection model, and three shallow learning methods: Extra-Tree, Random-Forest and Isolation-Forest. We used the results provided in [4] for the LSTM, Extra-Tree and Random-Forest models. However, for the AutoEncoder and Isolation-Forest models we had to conduct new experiments because the results in [4] are for the balanced version of datasets. In addition to the model performance results provided in [4], we also used an MLP (Feed Forward) model in this study, to provide an additional baseline result. This MLP model is similar to the MLP model utilised to create the DANN, i.e., a combination of the G_f and G_C blocks of the DANN, as shown in Fig. 3-(a). Selecting similar models in the DANN component of DI-NIDS and the baseline MLP allowed us to evaluate the role of the augmenting block G_D in the cross-domain evaluation. Tables 2 and 3 show the parameters of the conventional deep and shallow learning models used for the comparison.

In addition to these conventional ML models, we also included the OSVM and DANN blocks individually in our evaluation. This allowed us to evaluate the performance of DI-NIDS and compare it with the performance of its key building blocks separately. For the evaluation of OSVM and DANN, each of these models was separately trained and evaluated. In the case of domain-specific evaluation, these models were trained and tested on NFv2-UNSW-NB15 and NFv2-CIC-2018 datasets separately. In the case of the cross-domain evaluation, each model was once trained on NFv2-UNSW-NB15 and tested on NFv2-CIC-2018 and then trained on NFv2-CIC-2018 and tested against NFv2-UNSW-NB15.

Table 4 shows the results (F1-Score) of domain-specific performance evaluation for DI-NIDS, and the eight baseline ML models used for comparison. As can be seen, while the performance of all the models except AutoEncoder are largely similar on both NIDS datasets, DANN and DI-NIDS show the best performance on the NFv2-CIC-2018 and NFv2-UNSW-NB15 datasets respectively.

Table 2
The model parameters for three deep learning-based NIDSs along with the other parameters for training and evaluation of the models.

Parameter	Feed Forward	LSTM	AutoEncoder
No. Hidden layers	3	4	9
No. Nodes (each layer)	10	10	Different[a]
Learning rate	0.0001	0.0001	0.0001
Dropout ratio	0.2	0.2	0.2
Batch size	512	512	512
Validation split	0.3	0.3	0.3
No. of folds	5	5	5

[a]Nodes on layers [32,16,8,4,2,4,8,16,32].

Table 3
The model parameters for three shallow learning-based NIDSs along with the other parameters for training and evaluation of the models.

Parameter	Random-Forest	Extra Tree	Isolation-Forest
ccp_alpha	0.001	0.001	–
Batch size	512	512	512
Validation split	0.3	0.3	0.3
No. of folds	5	5	5

Table 4
Domain-specific performance (F1-Score (%)) of various ML models compared to DANN, OSVM and DI-NIDS when trained and evaluated on the same dataset.

ML model	NFv2-CIC-2018	NFv2-UNSW-NB15
Random-Forest [4]	95.44%	92.17%
Extra Tree [4]	84.62%	91.73%
LSTM NN [4]	90.17%	92.82%
Feed Forward NN[a]	97.72%	92.24%
OSVM	92.97%	98.28%
AutoEncoder	58.27%	75.52%
Isolation-Forest	91.02%	87.43%
DANN	**97.81**%	93.38%
DI-NIDS	93.23%	**98.68**%

[a]The feed forward neural network as depicted in Fig. 3-a.

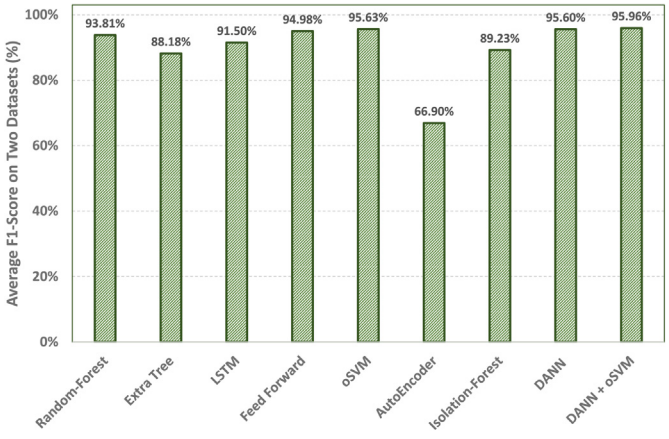


Fig. 7. Average domain-specific F1-Score of various ML models on two datasets NFv2-CIC-2018 and NFv2-UNSW-NB15 (shown in Table 4) where the test and training datasets are the same.

Fig. 7 shows the average domain-specific performance (F1-Score) of the considered models for the two datasets, as stated in Table 4. As can be seen, DI-NIDS has the highest average performance on these two datasets, closely followed by OSVM and DANN. While the main idea of proposing DI-NIDS is to address the low cross-domain performance of conventional ML-based NIDSs, the fact that DI-NIDS has the highest domain-specific performance among all the considered models is very promising.

In the next set of experiments, we compared the performance of DI-NIDS to the other models in two cross-domain setups. In this setting, first each model is trained on the NFv2-CIC-2018 dataset, and then tested against the NFv2-UNSW-NB15 dataset. Then the source and target domains are swapped, i.e., the models are trained on NFv2-UNSW-NB15 and tested against NFv2-CIC-2018. Tables 5 and 6 show the results of these evaluations respectively. Similar to the domain-specific evaluation, we used the results of the Random-Forest, Extra-Tree and LSTM models on the same datasets from [4] for cross-domain evaluation. For the Feed Forward model (MLP), AutoEncoder and Isolation-Forest, similar to the domain-specific evaluation, we used the same network setting/parameters as mentioned in Table 2.

In both tables, the first column shows the model name, the second column shows its cross-domain performance and the third column shows the difference between the cross-domain performance and its corresponding domain-specific evaluation. For instance, in Table 5 the Random-Forest model has a F1-Score of 0.84% when trained on NFv2-CIC-2018 and tested against NFv2-UNSW-NB15. This is 94.60% lower than its performance (F1-Score) when trained and tested on NFv2-CIC-2018, as shown in the first column of Table 4. Accordingly, the third column of

Table 5
Cross-domain performance (F1-Score (%)) and its difference to the corresponding domain-specific performance of DI-NIDS and conventional ML models for the case where source domain is **NFv2-CIC-2018** and the target domain is **NFv2-UNSW-NB15**.

ML model	F1-Score (%)	Performance degradation
Random-Forest [4]	0.84%	94.60%
Extra Tree [4]	0.57%	84.05%
LSTM NN [4]	9.63%	80.54%
Feed Forward NN[a]	3.09%	94.63%
OSVM	86.15%	6.79%
AutoEncoder	**86.27%**	**−28.00%**
Isolation-Forest	26.96%	64.06%
DANN	17.31%	80.50%
DI-NIDS	85.79%	7.44%

[a]The feed forward neural network as depicted in Fig. 3-a.

Table 6
Cross-domain performance (F1-Score (%)) and its difference to the corresponding domain-specific performance of DI-NIDS and conventional ML models for the case where source domain is **NFv2-UNSW-NB15** and the target domain is **NFv2-CIC-2018**.

ML model	F1-Score (%)	Performance degradation
Random-Forest [4]	7.70%	84.47%
Extra Tree [4]	17.47%	74.26%
LSTM NN [4]	14.20%	78.62%
Feed Forward NN[a]	30.79%	61.45%
OSVM	15.74%	82.54%
AutoEncoder	12.29%	63.23%
Isolation-Forest	32.80%	54.63%
DANN	61.94%	31.44%
DI-NIDS	**93.29**%	**5.39**%

[a]The feed forward neural network as depicted in Fig. 3-a.

Table 7
Cross-domain performance comparison degradation (Degr.) comparison.

Model	Train dataset	Test dataset	F1-Score (%)	Degr. (%)	Avg. Degr. (%)
Pontes et al. [15]	CIC-2017	CIC-2017	89.80	**11.10**	**19.30**
		CIC-2019	78.70		
	CIC-2019	CIC-2019	91.60	**27.50**	
		CIC-2017	64.10		
DI-NIDS	NFv2-CIC-2018	NFv2-CIC-2018	93.23	**7.44**	**6.42**
		NFv2-UNSW-NB15	85.79		
	NFv2-UNSW-NB15	NFv2-UNSW-NB15	98.68	**5.39**	
		NFv2-CIC-2018	93.29		

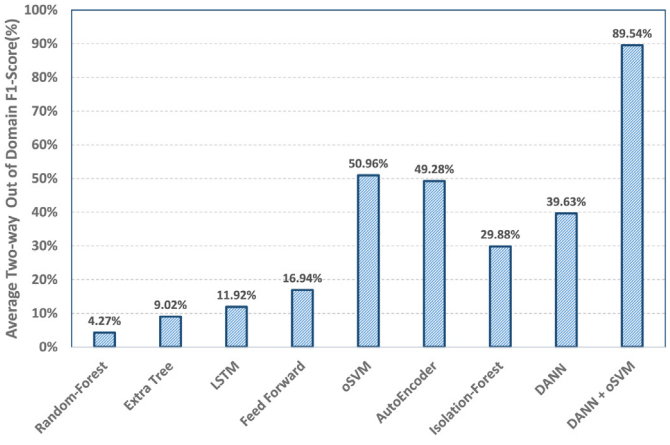


Fig. 8. Averaged cross-domain performance (F1-Score) of ML models across two cross-domain experiments.

Table 5 shows 94.60% for the performance degradation of the Random-Forest model.

As can be seen, AutoEncoder, which is closely followed up by one-class SVM, is the best performing model when NFv2-CIC-2018 is the source and NFv2-UNSW-NB15 is the target domain. In this case, the performance (F1-Score) of DI-NIDS is only 0.48% lower than AutoEncoder, i.e., 85.79%. While this is a great result for AutoEncoder in regards to domain adaptation, the next evaluation shows an entirely different results.

The results in Table 6 indicate that DI-NIDS is the best performing model in the second experiment, with a significant advantage over the other models. In fact, DI-NIDS is the only model capable of maintaining its performance over the two cross-domain experiments. It has a 7.44% performance degradation in the first cross-domain experiment and 5.39%. AutoEncoder, which was the best performing model in the first experiment, only achieves an F1-Score of 12.29% in this experiment, with 63.23% degradation compared to its corresponding domain-specific performance.

Fig. 8 shows the average performance (F1-Score) of the models for the two cross-domain experiments. As can be seen, all the conventional ML models, including Random-Forest, Isolation-Forest, Extra-Tree, LSTM and Feed Forward, have very low average cross-domain performance, no better than 29.88%. The OSVM, AutoEncoder and DANN models are somewhat better, with average cross-domain performances of 50.96%, 49.28% and 39.63% respectively. However, they are still far from a truly domain invariant model, i.e., a model that is capable of maintaining its performance in the presence of domain shifts. This is in clear contrast to DI-NIDS, which largely maintains its performance, with an average cross-domain performance of 89.54%.

5.4. Comparison to baseline results

While there is no previous work evaluating the cross-domain performance of NIDSs using the same set of datasets as used in this study, there is a previous work [15] running similar experiments using other datasets. The authors of [15] used two versions of the CIC-IDS dataset in their original format [13], CIC-2017 and CIC-2019, for their cross-domain evaluations. Although we cannot compare the absolute performance numbers, we believe it is possible to compare the corresponding degree of performance degradation from the domain-specific to the cross-domain evaluation scenario.

Table 7 shows the performances of DI-NIDS compared to the relevant state-of-the-art [15]. Here, each model is independently trained on two datasets, and evaluated on the training dataset (domain-specific evaluation), as well as the other, unseen dataset (cross-domain evaluation). The cross-domain results are shown as shaded cells in the table. The first column indicates the model, followed by the training dataset and the test dataset. The fourth column shows the corresponding F1-Scores.

The fifth column presents the *cross-domain degradation*, i.e. the difference between the F1-Score achieved in the domain-specific evaluation and the corresponding cross-domain evaluation. Finally, the sixth column shows the cross-domain degradation average across the two test and train dataset combinations.

While the absolute performance numbers of DI-NIDS and [15] in the table might not be comparable due the use of different datasets, we argue that it is reasonable to compare the corresponding cross-domain degradation figures. We can see that the cross-domain degradation values of DI-NIDS are 7.44% and 5.39% for the two 'directions' of evaluation, compared to 11.10% and 27.50% of [15].

One of the benefits of DI-NIDS is that its performance degradation is relatively consistent across the two directions of cross-domain evaluation, with only a (absolute) difference of 2% between the two values. In contrast, the model proposed in [15] is sensitive to the direction of cross-domain evaluation, and exhibits a higher degree of variance, with difference of more than 16% between the two directions of evaluation.

Furthermore, and more importantly, the average cross-domain degradation of DI-NIDS is only 6.42%, compared to 19.3% to the relevant state-of-the-art [15]. This is a significant improvement in terms of cross-domain performance, and represents a step towards more domain invariant, and hence practical ML-based NIDSs.

6. Conclusion

This paper proposes a *domain-invariant* network intrusion detection system (NIDS) framework to address the shortcomings of the existing NIDSs in regards to distribution shifts in data.

While domain adaptation methods to address the problem of distribution shift have been extensively studied in a range of machine learning application areas, it has not received significant attention in the context of ML-based NIDSs. This is despite the fact there are likely to be significant distribution shifts between training datasets and test data, in particular between (synthetic) training datasets and real-world production networks, such as shown in [1]. As we demonstrate in this paper, standard domain adaptation (DA) methods based on a supervised learning approach do not work very well for NIDSs. One of the key reasons is that the existing DA approaches generally assume balanced datasets. Realistic datasets (and network traffic in general) in the context of NIDS are highly imbalanced, with attack or anomalous traffic representing only a small proportion of the overall network traffic.

In order to address this gap, this paper proposes DI-NIDS, a domain invariant NIDS framework that takes into account the highly imbalanced nature of network traffic, while efficiently addressing the distribution shift between source and target domains. DI-NIDS achieves this by using a Domain-Adversarial Neural Network (DANN) to project the data into a domain-invariant feature space. The DANN is trained using data and labels from the source domain, and unlabelled data from the target domain. DI-NIDS learns features that discriminate between classes in the source domain, but that do not discriminate between the source and target domains. It leverages the feature extractor network of the trained DANN, and uses an OSVM model for the downstream task of traffic classification and anomaly detection. Our experimental results show that, in addition to achieving excellent domain-specific classification performance, DI-NIDS significantly improves the cross-domain performance over the relevant state-of-the-art. We believe that improving the domain invariance, and robustness against feature distribution shifts, for ML-based NIDSs is an important step towards a more widespread deployment of such systems in practical real-world networks.

CRediT authorship contribution statement

Siamak Layeghy: Conceptualization, Software, Validation, Investigation, Writing – original draft, Visualization. **Mahsa Baktashmotlagh:** Methodology, Validation, Investigation, Writing – review & editing. **Marius Portmann:** Validation, Investigation, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data used in this study is publicly available

Acknowledgements

This research is made possible by an Advance Queensland Industry Research Fellowship, grant number RM2019002409.

References

[1] S. Layeghy, M. Gallagher, M. Portmann, Benchmarking the benchmark - analysis of synthetic NIDS datasets, 2021, arXiv preprint arXiv:2104.09029.
[2] S.M. Erfani, M. Baktashmotlagh, M. Moshtaghi, V. Nguyen, C. Leckie, J. Bailey, K. Ramamohanarao, Robust domain generalisation by enforcing distribution invariance, in: IJCAI International Joint Conference on Artificial Intelligence, Volume 2016-Janua,2016, pp. 1455–1461.
[3] S. Al-riyami, F. Coenen, A. Lisitsa, A re-evaluation of intrusion detection accuracy : an alternative evaluation strategy, in: ACM SIGSAC Conference on Computer and Communications Security, 2018, pp. 2195–2197.
[4] S. Layeghy, M. Portmann, Explainable cross-domain evaluation of ML-based network intrusion detection systems, 2023, http://dx.doi.org/10.1016/j.compeleceng.2023.108692.
[5] Mohammad J. Hashemi, Detecting anomalies in network systems by leveraging neural networks, (Ph.D. thesis), University of Colorado, 2021.
[6] M. Baktashmotlagh, M.T. Harandi, B.C. Lovell, M. Salzmann, Unsupervised domain adaptation by domain invariant projection, in: Proceedings of the IEEE International Conference on Computer Vision, 2013, pp. 769–776.
[7] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, V. Lempitsky, U. Dogan, M. Kloft, F. Orabona, T. Tommasi, A. Ganin, Domain-adversarial training of neural networks, J. Mach. Learn. Res. 17 (2016) 1–35.
[8] B. Scholkopf, R. Williamson, A. Smola, J. Shawe-taylor, J. Platt, Support vector method for novelty detection, in: Advances in Neural Information Processing Systems, Vol. 12, 1999, pp. 1–7.
[9] D. Brauckhoff, A. Wagner, M. May, FLAME: A flow-level anomaly modeling engine, in: Terry Benzel (Ed.), Workshop on Cyber Security Experimentation and Test, USENIX Association, San Jose, California, USA - CSET, 2008, p. 6, http://www.usenix.org/events/cset08/tech/full%5C_papers/brauckhoff/brauckhoff.pdf.
[10] J.K. Samriya, R. Tiwari, X. Cheng, R.K. Singh, A. Shankar, M. Kumar, Network intrusion detection using ACO-DNN model with DVFS based energy optimization in cloud framework, Sustain. Comput.: Inform. Syst. 35 (2022) 100746.
[11] Y. Zhang, J. Yan, Semi-supervised domain-adversarial training for intrusion detection against false data injection in the smart grid, in: Proceedings of the International Joint Conference on Neural Networks, 2020.
[12] Y. Fan, Y. Li, H. Cui, H. Yang, Y. Zhang, An intrusion detection framework for IoT using partial domain adaptation, in: International Conference on Science of Cyber Security, Vol. 2, Springer International Publishing, 2021, pp. 36–50, http://dx.doi.org/10.1007/978-3-030-89137-4_3.
[13] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy 2018-January, 2018, pp. 108–116.
[14] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, Asaf Shabtai, Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: an ensemble of autoencoders for online network intrusion detection, 2018, arXiv preprint arXiv:1802.09089.
[15] C.F.T. Pontes, M.M.C.D. Souza, J.J.C. Gondim, M. Bishop, M.A. Marotta, A new method for flow-based network intrusion detection using the inverse potts model, IEEE Trans. Netw. Serv. Manage. 18 (2021) 1125–1136.
[16] O. Ajayi, A. Gangopadhyay, DAHID : Domain adaptive host-based intrusion detection, in: IEEE International Conference on Cyber Security and Resilience (CSR) Workshops DAHID, IEEE, 2021, pp. 467–472.
[17] A. Singla, E. Bertino, D. Verma, Preparing network intrusion detection deep learning models with minimal data using adversarial domain adaptation, in: ACM SIGSAC Conference on Computer and Communications Security, 2020, pp. 127–140.
[18] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Military Communications and Information Systems Conference (MilCIS), IEEE, IEEE, 2015, pp. 1–6, http://dx.doi.org/10.1109/MilCIS.2015.7348942.
[19] M. Tavallaee, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the KDD cup 99 data set, in: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009, pp. 1–6, http://dx.doi.org/10.1109/CISDA.2009.5356528.
[20] F. Ullah, S. Ullah, M.R. Naeem, L. Mostarda, S. Rho, X. Cheng, Cyber-threat detection system using a hybrid approach of transfer learning and multi-model image representation, Sensors 22 (2022) 5883.
[21] A.G. B, I. Odebode, Y. Yesha, A domain adaptation technique for deep learning in cybersecurity, in: OTM Confederated International Conferences on the Move To Meaningful Internet Systems, Vol. 1, Springer International Publishing, 2020, pp. 221–228, http://dx.doi.org/10.1007/978-3-030-40907-4_24.
[22] P. Wu, H. Guo, R. Buckland, A transfer learning approach for network intrusion detection, in: 4th IEEE International Conference on Big Data Analytics, 2019, http://dx.doi.org/10.1109/ICBDA.2019.8713213.
[23] Y.X. Yingying Xu, Zhi Liu, Yanmiao Li, Yushuo Zheng, Haixia Hou, Mingcheng Gao, Yongsheng Song, Y. Xu, Z. Liu, Y. Li, Y. Zheng, H.M.G. Hou, Y. Song, Y. Xin, Intrusion detection based on fusing deep neural networks and transfer learning, in: International Forum on Digital TV and Wireless Multimedia Communications. Vol. 1, Springer Singapore, 2020, pp. 212–221, http://dx.doi.org/10.1007/978-981-15-3341-9_18.
[24] KDD Cup 1999 Data, University of California, Irvine, 1999, http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html, Accessed: 2020-07-30.
[25] M. Long, Y. Cao, J. Wang, M.I. Jordan, Learning transferable features with deep adaptation networks, in: 32nd International Conference on Machine Learning, ICML 2015, Vol. 1, 2015, pp. 97–105.

[26] E. Tzeng, Judy Hoffman, Kate Saenko, Trevor Darrell, Adversarial discriminative domain adaptation, in: CVPR, InProceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 7167–7176.
[27] A. Gretton, K.M. Borgwardt, M.J. Rasch, B. Schökopf, A. Smola, A kernel two-sample test, J. Mach. Learn. Res. 13 (2012) 723–773.
[28] J. Hoffman, E. Tzeng, T. Park, J.Y. Zhu, P. Isola, K. Saenko, A.A. Efros, T. Darrell, CyCADA: Cycle-consistent adversarial domain adaptation, in: 35th International Conference on Machine Learning, ICML 2018, Vol. 5, 2018, pp. 3162–3174.
[29] B. Sun, K. Saenko, Deep CORAL: Correlation alignment for deep domain adaptation, in: Computer Vision – ECCV 2016 Workshops, Vol. 9915, 2016, pp. 443–450, http://dx.doi.org/10.1007/978-3-319-49409-835.
[30] G. Kang, L. Jiang, Y. Yang, A.G. Hauptmann, Contrastive adaptation network for unsupervised domain adaptation, in: Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2019, pp. 4893–4902, http://dx.doi.org/10.1109/CVPR.2019.00503.
[31] Q. Zhou, K.Y. Zhang, T. Yao, R. Yi, K. Sheng, S. Ding, L. Ma, Generative domain adaptation for face anti-spoofing, in: Computer Vision–ECCV 2022: 17th European Conference, Springer Nature Switzerland, 2022, pp. 335–356, http://dx.doi.org/10.1007/978-3-031-20065-6_20.
[32] V. Narayanan, Aniket An, Deshmukh, U. Dogan, V.N. Balasubramanian, On challenges in unsupervised domain generalization, in: Proceedings of Machine Learning Research, Vol. 181, 2022, pp. 42–58.
[33] P. Oza, H.V. Nguyen, V.M. Patel, Multiple Class Novelty Detection under Data Distribution Shift, in: LNCS, Vol. 12352, 2020, pp. 432–449.
[34] C. Cortes, V. Vapnik, Support-vector networks, Mach. Learn. 20 (1995) 273–297.
[35] R. Vlasveld, Introduction to one-class support vector machines, 2013, http://rvlasveld.github.io/blog/2013/07/12/introduction-to-one-class-support-vector-machines/.
[36] M. Sarhan, S. Layeghy, N. Moustafa, M. Portmann, Netflow datasets for machine learning-based network intrusion detection systems, 2020, arXiv preprint arXiv:2011.09144.
[37] M. Sarhan, S. Layeghy, N. Moustafa, M. Portmann, Towards a standard feature set of NIDS datasets, 2021, arXiv preprint arXiv:2101.11315.
[38] Y. Ganin, V. Lempitsky, Unsupervised domain adaptation by backpropagation, in: International Conference on Machine Learning, PMLR, 2015, pp. 1180–1189.
[39] J.B. Tenenbaum, V. De Silva, J.C. Langford, A global geometric framework for nonlinear dimensionality reduction, Science 290 (2000) 2319–2323.